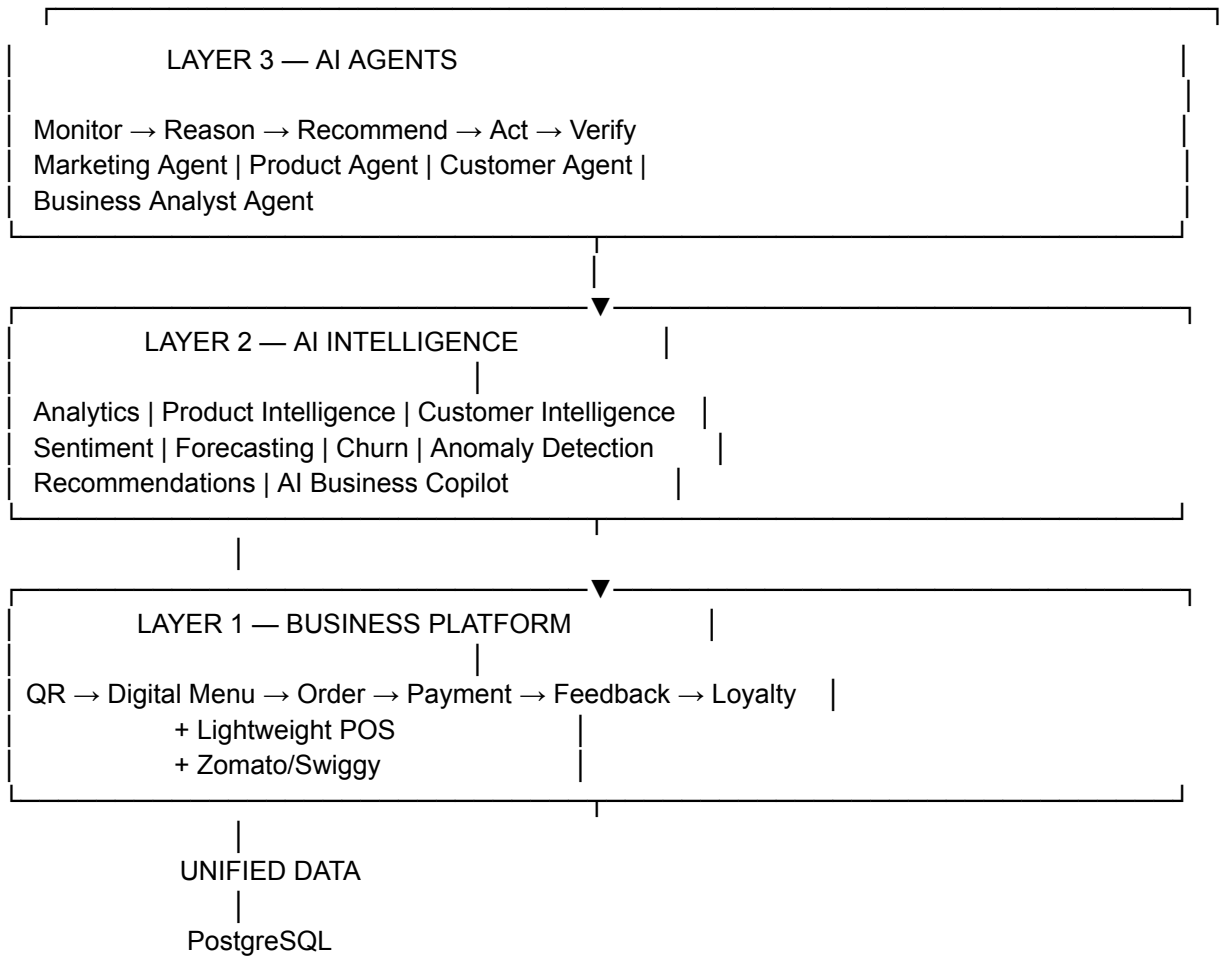
AI-Native Café Intelligence Platform

with three commercially independent layers



The most important principle:

Layer 1 creates the data. Layer 2 understands the data. Layer 3 acts on the data.

And because you want each layer to be sellable independently:

Customer A → Layer 1 only
Customer B → Layer 1 + Layer 2
Customer C → Layer 1 + Layer 2 + Layer 3

The architecture must support that from day one.

PART 0 — First, define exactly what you are building

Before the roadmap, let's establish the boundaries.

Layer 1 — Restaurant Transaction Platform

You are **not** building a complete restaurant ERP.

Layer 1 contains:

Customer side

- QR scanning
- Digital menu
- Product browsing
- Cart
- Ordering
- Customer identification
- Payment
- Feedback
- Loyalty/reward

Restaurant POS

- Login
- Product management
- Menu management
- Table management
- Order management
- Billing
- Payment status
- Order history
- Basic sales reports

External order ingestion

- Zomato
- Swiggy
- Later other channels

Data

Everything produces structured events.

Layer 2 — Intelligence

This layer consumes Layer 1 data.

It answers:

> What happened?
>
> What is happening?
>
> Why is it happening?
>
> What is likely to happen?
>
> What should the owner do?

Examples:

- Revenue analytics
- Product analytics
- Product sentiment
- Customer segmentation
- Repeat-purchase analysis
- Churn prediction
- Anomaly detection
- Product forecasting
- Customer behavior
- AI-generated insights
- Business dashboard
- Natural-language business questions

Layer 3 — AI Agents

Layer 2 **tells** the owner.

Layer 3 **does something about it**.

For example:

Layer 2:

"Repeat customers have declined 13%."

Layer 3:

"I identified 84 high-value customers who haven't visited in 21+ days."

Then:

"I prepared a WhatsApp campaign."

Eventually:

"Campaign sent to 84 customers."

Then:

"17 returned and generated ₹8,400."

That is an agent.

PART 1 — Overall development roadmap

For a three-person team, I would structure the project into roughly:

PHASE 0 Product & System Design
PHASE 1 Engineering Foundation
PHASE 2 Layer 1 — Customer Experience
PHASE 3 Layer 1 — POS
PHASE 4 Layer 1 — Payments + Loyalty + Feedback
PHASE 5 External Order Integrations
PHASE 6 Data Platform
PHASE 7 Layer 2 — Analytics
PHASE 8 Layer 2 — ML Intelligence
PHASE 9 Layer 2 — AI Copilot
PHASE 10 Production/MLOps
PHASE 11 Layer 3 — Agent Foundation
PHASE 12 Individual Agents
PHASE 13 Agentic Automation
PHASE 14 Pilot → Production → Scale

TEAM STRUCTURE

You have:

👨‍💻 Developer 1 — Frontend

Responsible for:

- Customer web application
- POS interface
- Owner dashboard
- Design system
- State management
- API integration
- Charts
- Agent UI

Developer 2 — Backend

Responsible for:

- FastAPI
- Database
- Authentication
- POS APIs
- Orders
- Payments
- Loyalty
- Feedback
- Integrations
- Background jobs
- Security
- Infrastructure

🤖 Developer 3 — ML/AI

Responsible for:

- Analytics
- Data pipelines
- Feature engineering
- NLP
- ML models
- Forecasting

- Recommendation engine
- AI insight engine
- LLM layer
- Agents
- MLOps

PHASE 0 — PRODUCT & SYSTEM DESIGN

Duration: 1–2 weeks

Do NOT code heavily yet.

This phase prevents months of rework.

Backend developer

Design:

Multi-tenant architecture

You are going to have:

Platform
↓
Restaurant
↓
Branch
↓
Users

Every important record needs a `restaurant_id`.

Example:

restaurants
users
products
tables
orders
customers
feedback

loyalty
must all be associated with the appropriate restaurant.

Frontend developer

Design three interfaces:

Customer

QR
↓
Menu
↓
Product
↓
Cart
↓
Checkout
↓
Feedback

POS
Login
↓
Orders
↓
Products
↓
Tables
↓
Billing

Owner
Dashboard
↓
Analytics
↓
Insights
↓
AI

ML developer

Define the **analytics contract**.

Before training a single model, decide:

What questions are we eventually going to answer?

For example:

Which products are growing?

Which products are declining?

Why is a product declining?

Which products generate repeat customers?

Which customers are likely to churn?

What are customers complaining about?

Which hours perform best?

What changed this week?

What should the owner do?

Then determine:

> **What data is required to answer each question?**

This is incredibly important.

Team learning — Phase 0

All three

Learn:

- System design fundamentals
- REST API architecture

- Git/GitHub workflow
- Database design
- Authentication
- Multi-tenancy
- Docker fundamentals
- CI/CD fundamentals

Backend

Deep dive:

- PostgreSQL
- SQLAlchemy
- FastAPI architecture
- API versioning
- authentication/authorization

Frontend

Deep dive:

- React architecture
- TanStack Query
- component architecture
- responsive design
- frontend state management

ML

Deep dive:

- analytical data modeling
- SQL
- statistics
- experimentation
- ML lifecycle

PHASE 1 — ENGINEERING FOUNDATION

Duration: 1–2 weeks

Build the skeleton.

Backend

Set up:

FastAPI

PostgreSQL

SQLAlchemy

Alembic

Redis

Docker

JWT

Pydantic

Pytest

DATABASE

restaurants

users

branches

products

categories

tables

customers

orders

order_items

payments

feedback

loyalty_accounts

loyalty_transactions

Frontend

Create:

Customer App

POS App

Owner Dashboard

Shared:

UI components

authentication

routing

API client

error handling

loading states

ML

Create:

ml/

├── data/

├── preprocessing/

├── features/

├── models/

├── evaluation/

├── inference/

└── experiments/

Set up:

MLflow

from the beginning.

But don't build complicated MLOps infrastructure yet.

Phase 1 dependency

Backend creates:

API contracts

Database schema

Authentication

Frontend can then consume them.

ML doesn't need to block anything yet.

PHASE 2 — LAYER 1 CUSTOMER EXPERIENCE

Duration: ~3–4 weeks

This is your first major vertical slice.

The customer should be able to:

SCAN QR

↓

OPEN MENU

↓

SELECT PRODUCT

↓

ADD TO CART

↓

PLACE ORDER

Frontend

Build:

QR landing

Every table has:

restaurant_id

branch_id

table_id

QR token

Customer scans.

Backend resolves:

QR token

↓

Restaurant

↓

Branch

↓

Table

Menu

Categories

Coffee

Tea

Food

Desserts

Products:

Image

Name

Description

Price

Availability

Cart

Cappuccino × 2

Brownie × 1

Subtotal

Taxes

Total

Backend

Build:

GET /menu

GET /products

POST /orders

GET /orders/{id}

Order lifecycle:

CREATED

↓

CONFIRMED

↓

PREPARING

↓

READY

↓

COMPLETED

You can simplify this initially.

ML developer

No ML yet.

Instead:

Define event tracking.

Every important event becomes data.

For example:

menu_opened

product_viewed

product_added

cart_created

order_created

order_completed

feedback_submitted

loyalty_redeemed

This is extremely important.

You're building your future intelligence dataset.

PHASE 3 — LIGHTWEIGHT POS

Duration: 3–4 weeks

Now build the restaurant side.

Remember:

Lightweight POS.

Not ERP.

POS features

Dashboard

Today's revenue

Orders

Average order value

Order screen

New order

Products

Quantity

Customer

Table

Payment

Product management

Create

Edit

Delete

Deactivate

Menu management

Table management

Order history

Basic billing

Frontend dependency

Backend needs to expose:

POST /pos/orders

GET /pos/orders

PATCH /pos/orders/{id}

Frontend builds POS UI around those contracts.

ML dependency

Now ML gets its first useful dataset:

orders

order_items

products

customers

timestamps

tables

Start creating:

Analytics tables/views

For example:

daily_sales

product_sales

customer_activity

PHASE 4 — PAYMENT + FEEDBACK + LOYALTY

Duration: 2–3 weeks

Now your original customer loop is complete.

QR

↓

Menu

↓

Order

↓

Payment

↓

Feedback

↓

Loyalty

Backend

Payment integration.

Use a payment provider rather than building payment infrastructure yourself.

Webhook:

Payment Provider

↓

/webhooks/payment

↓

Order → PAID

Feedback

Customer receives:

How was your experience?

Rating:

1 2 3 4 5

Then:

What did you like?

What could we improve?

Loyalty

Important rule:

> **Reward feedback, not positive feedback.**

Example:

Feedback submitted

↓

+1 loyalty point

ML developer

Now the first real AI/NLP component can start.

Build:

Sentiment pipeline

Input:

> "Coffee was too sweet and cold."

Output:

```
{
  "sentiment": "negative",
```

```
  "topics": [
    "sweetness",
    "temperature"
  ]
}
```

Store structured feedback.

PHASE 5 — ZOMATO/SWIGGY + EXTERNAL ORDERS

Duration: 3–6+ weeks

This should happen **after your own order flow works**.

Don't let external integrations block the MVP.

Backend

Build an abstraction:

```
OrderSource
   |
   ├── Internal QR
   ├── POS
   ├── Zomato
   └── Swiggy
```

Every source eventually becomes:

UnifiedOrder

Example:

source = zomato

external_order_id = XYZ

internal_product_id = 123

Product mapping

This becomes essential:

Internal Product

    |

    ├── Zomato ID

    └── Swiggy ID

Frontend

POS should show:

ALL ORDERS

QR

Counter

Zomato

Swiggy

The owner doesn't care where the order came from.

ML

Now your data becomes much richer:

QR

POS

Zomato

Swiggy

Start channel analytics:

Revenue by channel

Product by channel

Orders by channel

Customer behavior by channel

PHASE 6 — DATA PLATFORM

Duration: 2–3 weeks

This is one of the most important phases.

Your product now has enough data to build intelligence properly.

Data architecture

I'd use:

PostgreSQL

Operational tables

Analytics transformations

Analytics tables/views

ML feature datasets

Don't immediately introduce a huge data warehouse.

PostgreSQL is enough initially.

ML developer

Create a proper feature layer.

Examples:

Customer features

visit_count

days_since_last_visit

average_order_value

favorite_product

purchase_frequency

total_spend

Product features

orders_7d

orders_30d

revenue_7d

revenue_30d

rating

rating_change

repeat_purchase_rate

sentiment_score

Restaurant features

daily_revenue

weekly_revenue

order_growth

customer_growth

repeat_rate

PHASE 7 — LAYER 2 ANALYTICS

Duration: 3–4 weeks

Now you build the actual intelligence dashboard.

LEVEL 1 — Descriptive analytics

Answer:

> **What happened?**

Dashboard:

Revenue

Orders

AOV

Customers

Repeat customers

Product sales

Channel sales

Ratings

LEVEL 2 — Diagnostic analytics

Answer:

> **Why did it happen?**

Example:

Revenue ↓ 12%

Why?

Traffic stable

Orders ↓ 4%

Average order value ↓ 8%

Why?

High-value product sales ↓ 18%

Then:

Why?

Cold Coffee ↓ 27%

Then:

Why?

Negative feedback ↑

This is where your product starts becoming genuinely differentiated.

PHASE 8 — PRODUCT INTELLIGENCE

Duration: 3–5 weeks

This should become your flagship feature.

For every product:

Product

|

├── Sales

├── Revenue

├── Growth

├── Rating

├── Sentiment

├── Repeat purchase

├── Customer segments

├── Time trends

└── Channel performance

Product Intelligence Score

You could eventually derive a composite score from:

Sales growth

+

Margin/contribution if available

+

Rating

+

Sentiment

+

Repeat purchase

+

Demand trend

But don't blindly create arbitrary scores.

Every score needs an interpretable methodology.

Example

Cold Coffee

Sales ↓ 17%

Rating ↓ 0.4

Repeat purchase ↓ 21%

Negative reviews ↑ 32%

AI insight:

> **Cold Coffee requires attention.**

Then explain:

> Sales have fallen 17% despite overall café traffic remaining stable. Repeat purchase declined 21%, while complaints about sweetness increased significantly.

This is your first real **AI business insight**.

PHASE 9 — CUSTOMER INTELLIGENCE

Duration: 3–4 weeks

Build:

Customer segmentation

Examples:

New

Regular

High-value

At-risk

Churned

RFM

Recency

How recently did they visit?

Frequency

How often?

Monetary

How much do they spend?

Churn model

Start simple.

Features:

days_since_last_visit

visit_frequency

total_visits

spend

rating

favorite_product

Model:

Logistic Regression

Then experiment with:

XGBoost

Output:

Customer 123

Churn probability = 0.82

PHASE 10 — FORECASTING + ANOMALY DETECTION

Duration: 3–4 weeks

Now predict:

Product demand

Expected Cold Coffee orders next week

Revenue

Expected revenue next 7 days

Anomalies

Today's sales are statistically unusual.

Models

Start with simple baselines:

Moving average

Exponential smoothing

Then:

XGBoost

LightGBM

Time-series models

Don't use deep learning unless your data actually warrants it.

PHASE 11 — AI BUSINESS COPILOT

Duration: 2–3 weeks

Now put an LLM interface on top of the intelligence.

Owner asks:

> "Why did my revenue fall this week?"

System:

Question

↓

Intent detection

↓

Analytics tools

↓

SQL / metrics

↓

ML outputs

↓

Evidence

↓

LLM explanation

The LLM shouldn't invent numbers.

Example

Owner:

> "Which product is hurting my business?"

AI:

> "Cold Coffee is currently the product requiring the most attention."

Then evidence:

Sales ↓ 17%

Rating ↓ 0.4

Repeat purchase ↓ 21%

Negative sentiment ↑ 32%

Then:

> "The strongest negative feedback theme is sweetness."

This is your **Layer 2 AI Copilot**.

PHASE 12 — PRODUCTION MLOPS

This should happen progressively, not as a giant project at the end.

Now establish:

Data

↓

Validation

↓

Feature pipeline

↓

Training

↓

Evaluation

↓

MLflow

↓

Model registry

↓

Deployment

↓

Monitoring

MLOps stack

Use:

DVC

Dataset/version management where appropriate.

MLflow

- experiments
- metrics
- models
- model registry

Docker

Reproducible environments.

GitHub Actions

CI/CD.

Model monitoring

You need:

Performance monitoring

Is churn model still accurate?

Data drift

Are customers behaving differently?

Prediction drift

Are model outputs changing?

Model versioning

churn_v1

churn_v2

churn_v3

PHASE 13 — LAYER 3 AGENT FOUNDATION

This is where the product becomes substantially more ambitious.

An AI agent is not simply:

> ChatGPT + prompt.

Your agent needs:

Observe

↓

Understand

↓

Plan

↓

Use tools

↓

Take action

↓

Observe result

↓

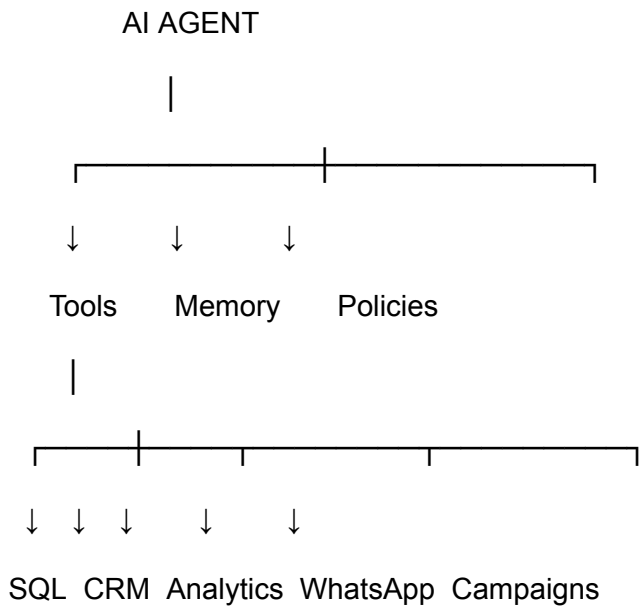
Evaluate

Agent architecture

OWNER

|

↓



Build an Agent Tool Layer

This is extremely important.

Don't allow agents to directly access your database.

Create tools:

get_revenue()

get_product_performance()

get_customer_segments()

get_churn_risk()

get_feedback()

create_campaign()

send_message()

get_campaign_results()

Now the agent can reason through controlled interfaces.

PHASE 14 — FIRST AGENT: BUSINESS ANALYST

This should be your first agent.

Why?

Because it's closest to Layer 2.

Owner:

> "What happened this week?"

Agent:

get_revenue()

get_orders()

get_product_performance()

get_customer_metrics()

get_feedback()

Then reasons.

Output:

> "Revenue increased 7%, primarily because..."

PHASE 15 — PRODUCT INTELLIGENCE AGENT

Owner:

> "Check my products."

Agent examines:

sales

ratings

sentiment

repeat purchase

trend

Then:

> "Three products require attention."

It explains why.

PHASE 16 — CUSTOMER RETENTION AGENT

Agent identifies:

high-value customers

+

at-risk customers

Example:

> "84 high-value customers haven't returned within their normal visit interval."

Then:

> "Would you like me to create a campaign?"

Owner approves.

Agent creates it.

PHASE 17 — MARKETING AGENT

Now the agent can:

Identify segment

↓

Create campaign

↓

Generate message

↓

Estimate audience

↓

Request approval

↓

Send

↓

Track

Example:

84 customers

Campaign:

"We've missed you..."

Then measure:

sent

delivered

clicked

visited

purchased

revenue generated

Now your agent learns from outcomes.

PHASE 18 — AUTONOMOUS AGENTS

This is the final level.

Instead of:

> Owner asks.

Agent continuously monitors.

Every morning

↓

Analyze business

↓

Detect anomalies

↓

Identify opportunities

↓

Generate recommendation

↓

Notify owner

Eventually:

> "Your Cold Coffee rating dropped significantly over the past 7 days. I've identified the main complaint themes and prepared a customer campaign. Review it?"

That's genuinely agentic.

THE THREE-LAYER PRODUCT MODEL

This is how I'd package it commercially.

PACKAGE 1 — OPERATE

Layer 1

QR

Menu

Ordering

POS

Payments

Feedback

Loyalty

Target:

> Small/new cafés that mainly need digital operations.

PACKAGE 2 — INTELLIGENCE

Layer 1 +

Layer 2

Analytics

Product Intelligence

Customer Intelligence

AI Insights

Forecasting

Churn

AI Copilot

Target:

> Café owners who want to understand and improve the business.

PACKAGE 3 — AUTONOMOUS

Layer 1 +

Layer 2 +

Layer 3

Business Analyst Agent

Product Agent

Customer Agent

Marketing Agent

Monitoring Agent

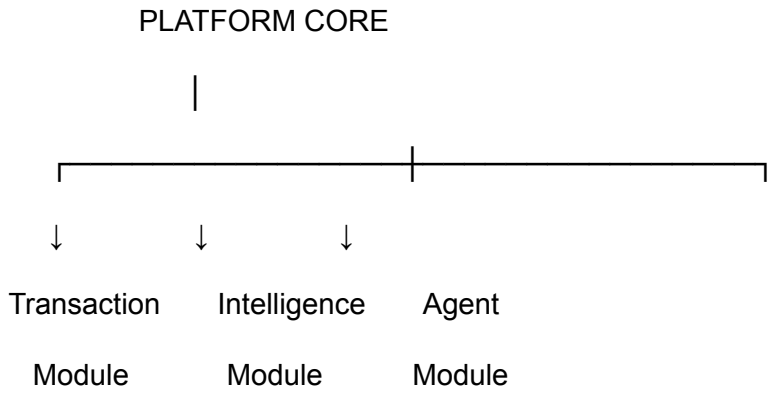
Target:

> Growth-focused cafés/chains.

VERY IMPORTANT: Architecture for independent layers

Don't literally build three separate applications.

Build:



And licensing:

restaurant.features

layer_1 = true

layer_2 = false

layer_3 = false

or:

layer_1 = true

layer_2 = true

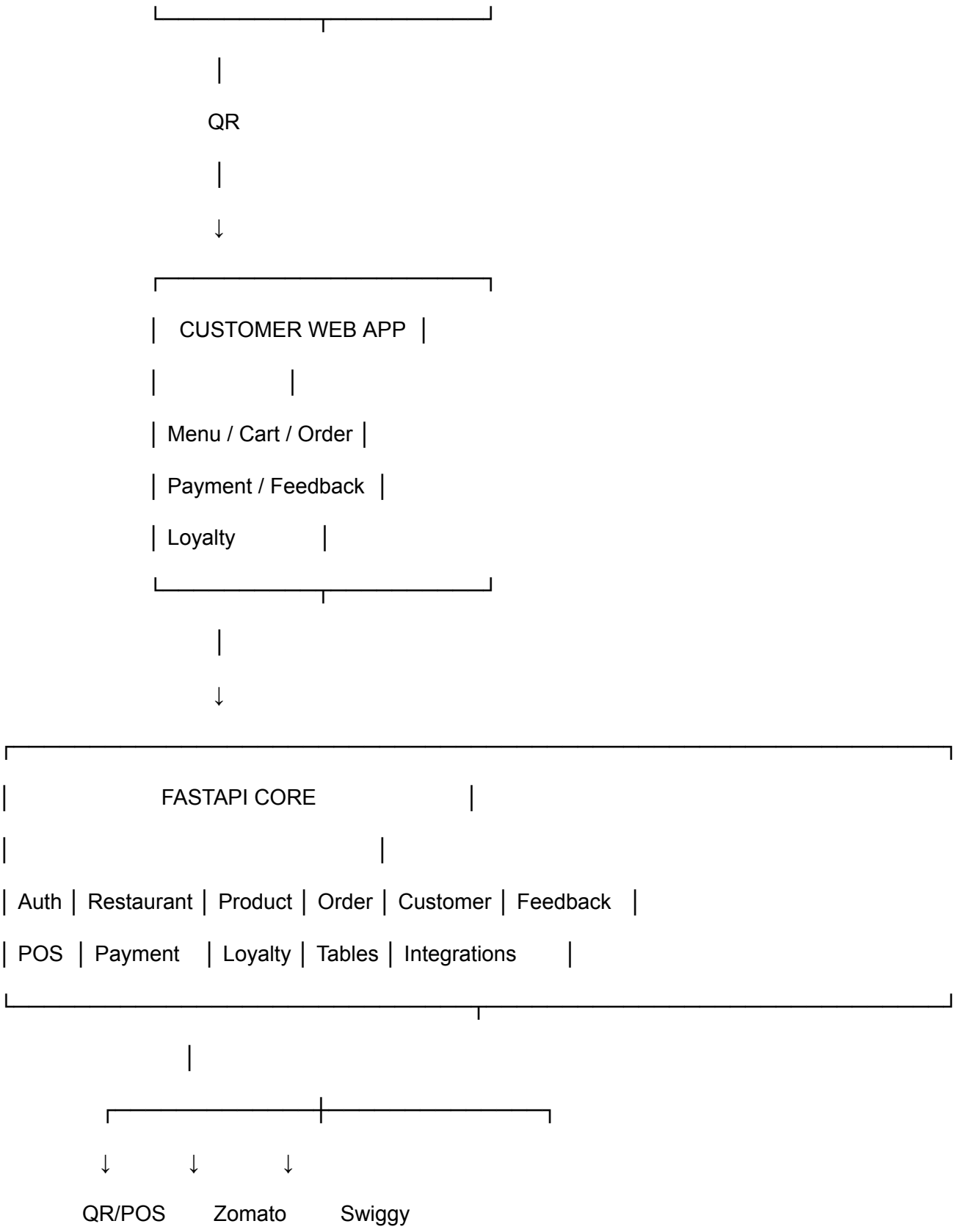
layer_3 = true

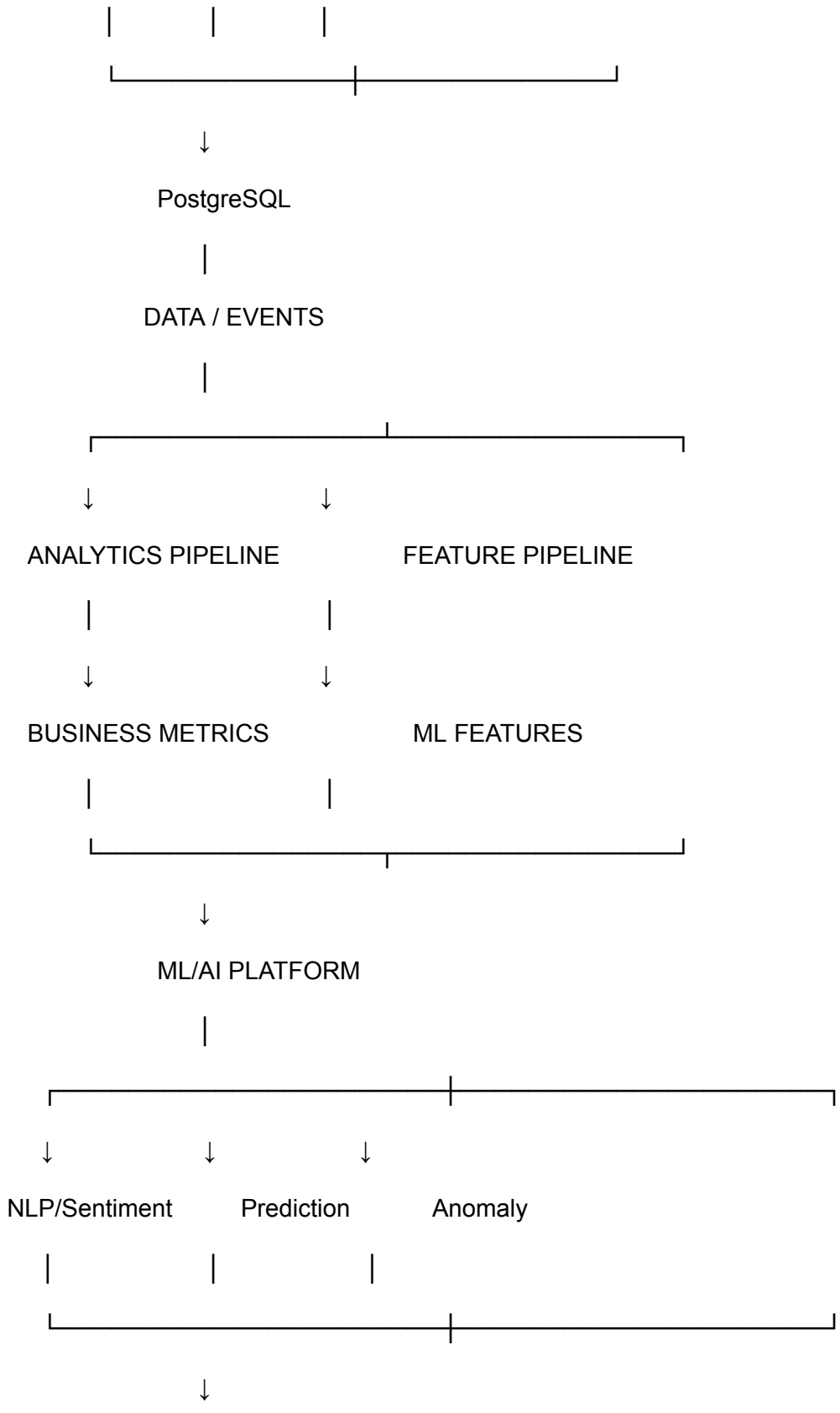
This allows you to turn capabilities on/off.

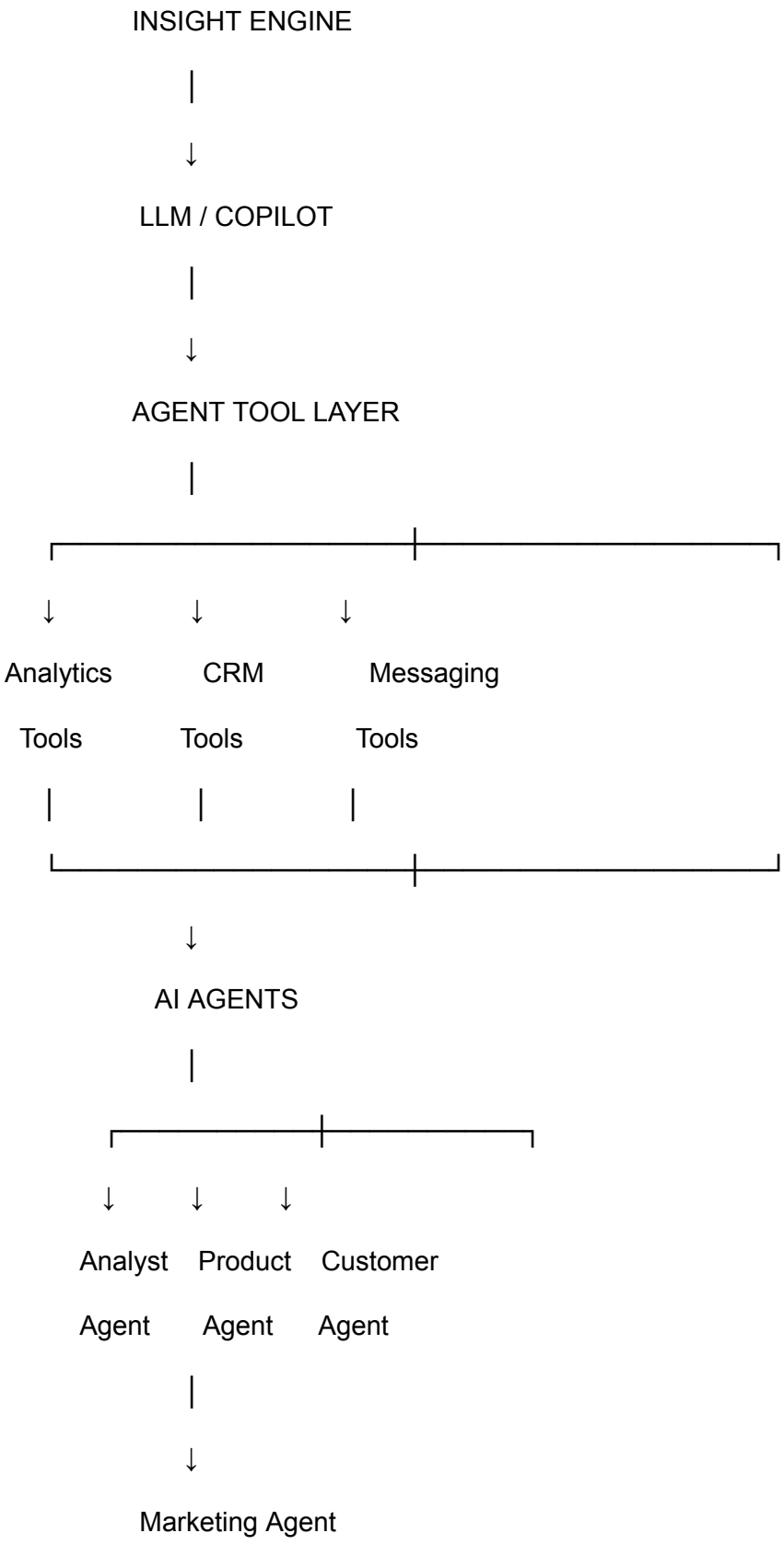
FINAL SYSTEM ARCHITECTURE

This is the architecture I'd want the team to eventually reach:




QR
CUSTOMER WEB APP
Menu / Cart / Order
Payment / Feedback
Loyalty
FASTAPI CORE
Auth
Restaurant
Product
Order
Customer
Feedback
POS
Payment
Loyalty
Tables
Integrations
QR/POS
Zomato
Swiggy


PostgreSQL
DATA / EVENTS
ANALYTICS PIPELINE
FEATURE PIPELINE
BUSINESS METRICS
ML FEATURES
ML/AI PLATFORM
NLP/Sentiment
Prediction
Anomaly


INSIGHT ENGINE
LLM / COPILOT
AGENT TOOL LAYER
Analytics Tools
CRM Tools
Messaging Tools
AI AGENTS
Analyst Agent
Product Agent
Customer Agent
Marketing Agent

THE THREE DEVELOPERS' COMPLETE RESPONSIBILITY MAP

Here's the part I'd actually give your team.

Phase	Frontend	Backend	ML/AI
Product design	UX flows	architecture	data requirements
Foundation	React architecture	FastAPI + DB	ML architecture
QR/Menu	Customer UI	menu APIs	event definitions
Ordering	Cart/order UI	order engine	analytics schema
POS	POS UI	POS backend	sales metrics
Payments	checkout	payment/webhooks	—
Feedback	review UI	feedback APIs	sentiment
Loyalty	loyalty UI	loyalty engine	customer features
Zomato/Swiggy	order display	integrations	channel analytics

Data platform	analytics UI	data APIs	feature pipeline
Analytics	charts/dashboard	aggregation APIs	statistical analysis
Product intelligence	product insights UI	insight APIs	product models
Customer intelligence	customer UI	customer APIs	churn/segmentation
Forecasting	forecast charts	model serving	forecasting
AI Copilot	chat interface	AI gateway/tools	reasoning/LLM
MLOps	model result UI	serving/infrastructure	MLflow/DVC/monitoring
Agent foundation	agent UI	tool APIs/permissions	agent orchestration
Agents	approval/action UI	action APIs	agent logic
Automation	notifications	scheduler/webhooks	monitoring/reasoning
Production	frontend deployment	DevOps/security	MLOps

DEPENDENCY MAP

This is critical.

Phase 1

Backend

↓

Database + API contracts

↓

Frontend

Phase 2

Backend

↓

Order events

↓

ML

Frontend and backend can work simultaneously.

Phase 3

Backend

↓

Structured transaction data

↓

ML analytics

↓

Frontend dashboard

Phase 4

Backend

↓

Feedback data

↓

NLP

↓

Backend insight API

↓

Frontend

Phase 5

External integrations

↓

Unified data model

↓

Analytics

↓

ML

↓

Dashboard

Phase 6

Layer 2 data

↓

Insight Engine

↓

LLM

↓

Copilot UI

Phase 7

Layer 2 tools

↓

Agent

↓

Backend action APIs

↓

External actions

↓

Results

↓

Agent

That last feedback loop is what makes the agents genuinely useful.

WHAT EACH PERSON NEEDS TO LEARN

Frontend developer

Stage 1

Learn deeply:

- React architecture
- component design
- routing
- TanStack Query
- form handling
- authentication

Stage 2

Learn:

- data visualization
- Recharts
- dashboard UX
- real-time updates
- WebSockets/SSE

Stage 3

Learn:

- agent UI
- streaming responses
- approval workflows
- optimistic updates
- notification UX

Backend developer

Stage 1

Master:

- FastAPI
- PostgreSQL
- SQLAlchemy
- REST
- JWT
- RBAC

Stage 2

Learn:

- system design
- caching
- Redis
- background jobs
- webhooks
- payment integration

Stage 3

Learn:

- API gateways
- event-driven architecture
- idempotency
- distributed systems
- multi-tenancy
- security

Stage 4

Learn:

- Docker
- CI/CD
- AWS
- monitoring
- logging
- secrets management

Stage 5

Learn:

- agent tool architecture
- permission systems
- action auditing
- rate limits
- human-in-the-loop systems

ML/AI developer

This person has the biggest learning progression.

Stage 1

Learn:

- SQL
- statistics
- analytics
- data modeling
- experiment design

Stage 2

Learn:

- NLP
- sentiment analysis
- topic extraction
- embeddings

Stage 3

Learn:

- feature engineering

- churn prediction
- classification
- regression
- clustering
- anomaly detection
- forecasting

Stage 4

Learn:

- MLflow
- DVC
- model registry
- model serving
- data validation
- drift detection

That's your **MLOps stage**.

Stage 5

Learn:

- LLM architecture
- RAG
- structured outputs
- function/tool calling
- evaluation
- prompt engineering
- LLM observability

Stage 6

Learn:

- agent architecture
- planning

- tool use
- memory
- state machines
- guardrails
- human approval
- agent evaluation

DEVOPS ROADMAP

Don't wait until the end.

Month 1

Git

GitHub

Docker

.env

Month 2

Docker Compose

CI

Automated tests

Month 3

CD

Cloud deployment

HTTPS

Secrets

Month 4+

Monitoring

Logging

Alerts

Backups

Later

Scaling

Load balancing

Caching

Queues

Observability

MLOPS ROADMAP

Similarly:

Raw Data

↓

Data validation

↓

Feature engineering

↓

Training

↓

MLflow experiment

↓

Evaluation

↓

Model registry

↓

Deployment

↓

Monitoring

Don't build all of this for the first sentiment classifier.

Introduce the process progressively.

THE MOST IMPORTANT DEVELOPMENT PRINCIPLE

Build vertical slices.

Don't spend three months doing:

> Backend first.

Then:

> Frontend.

Then:

> ML.

Instead:

Slice 1

QR

↓

Menu

↓

Order

↓

Database

↓

POS sees order

Working product.

Then:

Slice 2

Order

↓

Payment

↓

Feedback

↓

Database

Then:

Slice 3

Feedback

↓

Sentiment

↓

Product analytics

↓

Dashboard

Then:

Slice 4

Sales

+

Feedback

+

Customer behavior

↓

AI insight

↓

Owner

Then:

Slice 5

AI insight

↓

Agent

↓

Action

↓

Result

This way, you have something working **at every stage**.

YOUR FIRST REAL MVP

I would define MVP-1 as:

CUSTOMER

QR

↓

MENU

↓

ORDER

↓

PAY

↓

FEEDBACK

↓

LOYALTY

↓

POS

↓

Sales Dashboard

At this point you can put it into a real café.

MVP-2

Add:

Product Analytics

Customer Analytics

Feedback Sentiment

Review Topics

Repeat Purchase

Now you can say:

> **Layer 2 is available.**

MVP-3

Add:

AI Insights

AI Copilot

Churn

Forecasting

Recommendations

MVP-4

Add:

Business Analyst Agent

Product Agent

Customer Agent

Marketing Agent

MVP-5 — FINAL PRODUCT

The owner logs in and sees:

GOOD MORNING ☕

Your café generated ₹82,400

this week.

↑ 8.4%

⚠️ 3 things need your attention

1. Cold Coffee sales ↓ 18%
2. Repeat customers ↓ 7%
3. Saturday revenue ↓ 12%

AI EXPLANATION

Cold Coffee's decline is strongly associated

with an increase in complaints about sweetness.

RECOMMENDED ACTION

Run a 7-day recipe experiment.

AI AGENT

I identified 84 high-value customers who

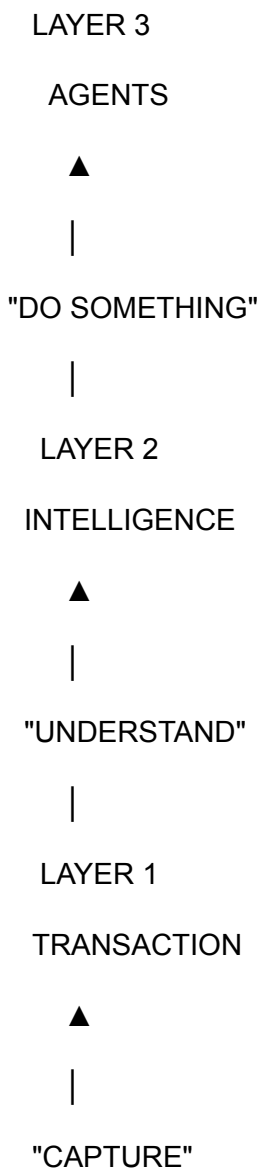
haven't returned within their normal interval.

[Create Campaign]

And that's when I would say you've achieved the vision.

One final architectural principle

Your product should evolve like this:



Layer 1 captures reality.

Layer 2 understands reality.

Layer 3 changes reality.

That should be the **central philosophy of the entire product**.

And because you're only three developers, I'd resist the temptation to build a massive microservices architecture. **FastAPI + PostgreSQL + Redis + background workers + React, organized as a modular monolith, is more than enough initially.** As the product earns real customers and specific bottlenecks appear, you split services based on evidence rather than architecture fashion.

The first major milestone I would set for the team is therefore **not "build Layer 1."** It is:

> **Build one complete vertical slice: `QR → Menu → Order → Payment → Feedback → Database → POS → Basic Insight`.**

Once that loop works with a real café's data, you've established the foundation on which all three layers can be built.